Università degli Studi di Milano
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
GIOVANNI DEGLI ANTONI

CORSO DI LAUREA TRIENNALE IN
INFORMATICA

IMPLEMENTAZIONE DIGITALE DI UN GIOCO DA TAVOLO CON GRAFICA INTERATTIVA PER IL POTENZIAMENTO DELLE COMPETENZE SOCIALI DI ADOLESCENTI AUTISTICI

Relatore: Prof. Alberto Borghese

Elaborato Finale di:
Samuel De Benedictis
Matr. Nr. 855264

Anno Accademico 2025-2026

Indice

Indice **i**

1 Introduzione **1**
1.1 L'autismo . . . 1
1.2 Il CFPIL di Varese . . . 2
1.3 Competenze sociali negli adolescenti autistici . . . 2
1.4 Il gioco da tavolo . . . 3
1.5 *La Città degli Imprevisti* . . . 3

2 Stato dell'arte **5**

3 Tecnologie utilizzate **6**
3.1 Framework e librerie di sviluppo . . . 6
3.1.1 Next.js . . . 6
3.1.2 TypeScript . . . 6
3.1.3 React . . . 7
3.1.4 Zustand . . . 7
3.1.5 Tailwind CSS . . . 7
3.2 Strumenti di qualità del codice . . . 7
3.2.1 Biome . . . 7
3.2.2 Husky e Commitlint . . . 7
3.3 Strumenti di testing e documentazione . . . 8
3.3.1 Vitest . . . 8
3.3.2 Playwright . . . 8
3.3.3 Storybook . . . 8

4 Implementazione **9**
4.1 Specifiche funzionali . . . 9
4.1.1 Configurazione della partita . . . 9
4.1.2 Svolgimento del turno . . . 9
4.1.3 Salvataggio e ripristino della partita . . . 10

4.2 Descrizione funzionale . . . 10
4.2.1 Schermata Home — Configurazione della partita . . . 10
4.2.2 Schermata di gioco . . . 10
4.2.3 Schermata Istruzioni . . . 12
4.2.4 Schermata Modalità avanzata . . . 12
4.2.5 Schermata Ripristina partita . . . 12
4.2.6 Schermata Feedback . . . 12
4.3 Architettura dei moduli . . . 12
4.3.1 Domain Model . . . 13
4.3.2 State Management . . . 13
4.3.3 Presentation Layer . . . 14
4.4 Scelte implementative rilevanti . . . 15
4.4.1 Il tipo `GameActionResult` e il Command Pattern . . . 15
4.4.2 Gestione dello stato con Zustand e persistenza . . . 16
4.5 Design pattern utilizzati . . . 17
4.5.1 Command Pattern . . . 17
4.5.2 Facade Pattern . . . 17
4.5.3 Builder Pattern . . . 18
4.5.4 Observer Pattern . . . 18
4.5.5 Manager Pattern . . . 18
4.5.6 Model-View-Controller . . . 18

5 Test **19**

6 Conclusioni **20**

Bibliografia **21**

Capitolo 1

Introduzione

1.1 L'autismo

"Autistico" è un aggettivo coniato dallo psichiatra svizzero Eugen Bleuler per descrivere la chiusura in sé stessi di alcuni suoi pazienti (dal greco *autos*) e in seguito viene utilizzato da molti autori per descrivere difficoltà osservate in pazienti bambini e adolescenti.

La psichiatra Grunya Efimovna Sukhareva nel 1926 descrive dettagliatamente le caratteristiche autistiche di sei bambini, che tuttavia vengono diagnosticati come affetti da psicopatia schizoide.

Solo nel 1943 viene proposta una distinzione dell'autismo da altre sindromi. Il pediatra Hans Asperger, nella sua tesi di dottorato, utilizza il termine *"psicopatia autistica"* e descrive accuratamente la condizione di alcuni ragazzi basandosi sull'osservazione di circa duecento pazienti. Leo Kanner, nello stesso anno, riporta le osservazioni compiute su undici bambini caratterizzate da *"marcate difficoltà nelle relazioni sociali e nella comunicazione*", e in un lavoro successivo propone di chiamare questa sindrome *"autismo infantile"*.

Negli anni Settanta, Lorna Wing fornisce le prime prove riguardo alla natura composita dell'autismo e l'impossibilità di descriverlo in base a criteri diagnostici univoci ed introduce l'espressione "disturbo dello spettro autistico".

Come riportato da Luca Surian, il termine *"spettro"* mette in risalto la concezione dimensionale che era stata proposta da Asperger, questa prospettiva sottolinea la variabilità quantitativa e graduale, piuttosto che la differenza qualitativa e categoriale; secondo questa prospettiva, la differenza tra persone con o senza autismo va pensata come una differenza di grado, non di qualità. [1]

L'autismo viene diagnosticato quando una costellazione di tratti è presente nello stesso individuo e si manifesta in forma molto accentuata. Alcuni di questi tratti sono: propensione per la ripetizione di alcune attività, spiccata attenzione ai dettagli e difficoltà nell'accettare cambiamenti. Queste caratteristiche, pur variando per intensità e manifestazione, rendono il potenziamento delle competenze sociali un obiettivo prioritario, al

quale il presente elaborato intende contribuire attraverso l'implementazione digitale di un gioco da tavolo strutturato.

1.2 Il CFPIL di Varese

Il *Centro Formazione Professionale Integrazione Lavorativa* (CFPIL), istituito dalla Provincia di Varese, rappresenta un punto di riferimento consolidato da oltre trent'anni per l'utenza disabile intellettiva, fisica e sensoriale. Nel 2002, il centro è stato integrato all'interno dell'"Agenzia Formativa della Provincia di Varese", divenuta successivamente Azienda Speciale nel dicembre 2009.

I progetti del CFPIL costituiscono una risposta strutturata alle esigenze specifiche di adolescenti, giovani e adulti con disabilità, offrendo percorsi formativi mirati e successivi inserimenti lavorativi.

L'attività formativa è progettata e monitorata attraverso équipe multidisciplinari composte da educatori professionali, psicologi e assistenti sociali. Questo approccio consente di predisporre curricoli formativi personalizzati, alcuni dei quali si realizzano in contesti lavorativi all'interno di realtà produttive, aziende private ed enti pubblici.

Il CFPIL svolge una funzione di mediazione tra le persone con disabilità e il mondo del lavoro, erogando un sistema integrato di servizi che comprende:

- Informazione, valutazione e orientamento professionale;
- Formazione professionale specializzata;
- Integrazione lavorativa;
- Monitoraggio post-assunzione.

Il centro promuove relazioni sistemiche con il tessuto sociale, scolastico, produttivo e istituzionale del territorio, al fine di definire e realizzare progetti formativi condivisi. Attraverso questa rete collaborativa, il CFPIL intende contribuire alla diffusione del principio della diversità come risorsa individuale e allo sviluppo di una "cultura della diversità" fondata sul rispetto e sull'inclusione. [2]

1.3 Competenze sociali negli adolescenti autistici

Il potenziamento delle competenze sociali degli utenti del CFPIL costituisce uno degli obiettivi primari dell'istituto, riconosciuto come condizione imprescindibile per favorire l'inserimento lavorativo e la partecipazione attiva alla vita comunitaria.

Per molti adolescenti con disturbo dello spettro autistico le competenze sociali rappresentano un'area di significativa difficoltà. Con questo termine si fa riferimento alla

capacità di interagire in modo adeguato al contesto, di comunicare bisogni e intenzioni, di rispettare turni e ruoli e di gestire situazioni di conflitto: abilità che risultano spesso più determinanti delle competenze tecniche ai fini dell'integrazione lavorativa.

Il CFPIL persegue questo obiettivo principalmente attraverso laboratori pratici a carattere professionalizzante, nei quali l'attività manuale diventa il mezzo attraverso cui esercitare e consolidare abilità relazionali in contesti strutturati e realistici, con ruoli definiti e procedure sequenziali che riproducono le dinamiche del mondo del lavoro.

Questo approccio laboratoriale permette di sviluppare le competenze sociali non attraverso esercizi astratti ma in situazioni autentiche e motivanti, favorendo la generalizzazione degli apprendimenti al di fuori del contesto scolastico o terapeutico.

1.4 Il gioco da tavolo

Per i bambini e gli adolescenti con disturbo dello spettro autistico, il gioco riveste un'importanza terapeutica significativa: rappresenta una cornice educativa motivante in cui sperimentare abilità, consolidare comportamenti e relazionarsi con le persone circostanti. Tra le varie forme di gioco, quella che raggiunge il più alto livello di strutturazione è il gioco da tavolo, caratterizzato dall'utilizzo di elementi fisici definiti (pedine, carte, tabelloni, dadi) e da un insieme di regole esplicite che delimitano uno spazio e un tempo di gioco ben determinati. Questa natura di sistema chiuso e altamente strutturato lo rende particolarmente adatto al contesto autistico, poiché presenta caratteristiche analoghe a quelle delle attività di lavoro indipendente previste dalla metodologia *Treatment and Education of Autistic and related Communication handicapped Children* (TEACCH): entrambe si svolgono seduti al tavolo, sono strutturate nel tempo e nello spazio, hanno una fine prestabilita al raggiungimento di condizioni note e privilegiano il codice visivo rispetto a quello verbale [3].

A differenza delle attività di lavoro individuale, tuttavia, il gioco da tavolo richiede la partecipazione di almeno due soggetti: si configura quindi come un'interazione focalizzata faccia a faccia, con implicazioni relazionali ed educative di rilievo.

Il progetto descritto nel presente elaborato si inserisce in questa prospettiva: l'implementazione digitale di un gioco da tavolo consente di integrare meccanismi di adattamento direttamente nell'interfaccia, senza modificare fisicamente i componenti, e offre al contempo uno strumento di interazione sociale strutturata per gli adolescenti del CFPIL di Varese.

1.5 *La Città degli Imprevisti*

Nel contesto delle attività del CFPIL di Varese, utenti e docenti hanno collaborato alla progettazione di un gioco da tavolo originale denominato *La Città degli Imprevisti.* Il gioco

si ispira alla struttura del Gioco dell'Oca, un percorso lineare con caselle numerate su cui i giocatori avanzano lanciando un dado, arricchendola però con caselle speciali che attivano minigiochi a carattere sociale ed emotivo. L'obiettivo di ogni partita è raggiungere l'ultima casella del tabellone prima degli avversari; lungo il percorso i giocatori si trovano a fronteggiare situazioni che richiedono comunicazione, collaborazione e riconoscimento delle emozioni.

Le caselle speciali costituiscono il cuore educativo del gioco e sono progettate per sollecitare competenze diverse:

- **Mimo**: il giocatore deve mimare una parola o una frase; allena la comunicazione non verbale e la capacità di decodificare i gesti altrui.
- **Quiz**: viene posta una domanda a risposta aperta o a scelta multipla; stimola l'attenzione e il ragionamento.
- **Parole alle spalle**: il giocatore scrive una parola sulla schiena di un compagno, che deve riconoscerla; promuove il contatto fisico guidato e la comunicazione tattile.
- **Disegno dettato**: un giocatore descrive a parole un'immagine che un compagno deve riprodurre graficamente; esercita la comunicazione verbale precisa e la collaborazione.
- **Emozioni in musica**: il giocatore esprime un'emozione attraverso una canzone; sviluppa il riconoscimento e l'espressione delle emozioni.
- **Indovina l'emozione**: viene mostrato un volto e il giocatore deve identificare l'emozione espressa; allena la lettura delle espressioni facciali, spesso difficoltosa nei soggetti con autismo.
- **Cosa faresti se...**: il giocatore risponde a uno scenario ipotetico e gli altri valutano la risposta; stimola il ragionamento sociale e la prospettiva altrui.
- **Test fisico**: viene proposta una prova motoria da eseguire; favorisce la coordinazione e la gestione del corpo nello spazio condiviso.
- **Battaglia**: quando due giocatori si trovano sulla stessa casella devono sfidarsi; introduce la competizione regolamentata e la gestione dell'esito (vittoria e sconfitta).
- **Caselle movimento**: alcune caselle fanno avanzare o retrocedere il giocatore; replicano la meccanica classica del Gioco dell'Oca, introducendo l'elemento della fortuna e dell'attesa.

La varietà delle sfide garantisce che durante la partita vengano esercitate competenze eterogenee, riducendo la ripetitività e mantenendo alto il coinvolgimento dei partecipanti.

Capitolo 2

Stato dell'arte

Capitolo 3

Tecnologie utilizzate

L'applicazione è sviluppata in TypeScript con il framework Next.js, a sua volta basato su React. Il processo di build produce un insieme di file statici (HTML, JavaScript, CSS) interpretabili da qualsiasi browser moderno, senza necessità di un server applicativo. Gli strumenti adottati sono stati scelti per garantire qualità del codice, efficienza di sviluppo e manutenibilità nel lungo periodo.

3.1 Framework e librerie di sviluppo

3.1.1 Next.js

Next.js è un framework React per applicazioni web che integra nativamente routing basato su file, rendering lato server (SSR) e generazione di siti statici (SSG). È stato scelto come framework principale per la sua capacità di produrre output statici distribuibili su GitHub Pages senza necessità di un server, mantenendo al contempo un'esperienza di sviluppo moderna.

3.1.2 TypeScript

TypeScript è un superset di JavaScript che aggiunge tipizzazione statica al linguaggio, consentendo di individuare errori a tempo di compilazione anziché a runtime. Nel contesto di un progetto con un domain model articolato — composto da classi, manager e interfacce — la tipizzazione ha reso più sicuro il refactoring e ha ridotto significativamente la categoria di bug legati a tipi inattesi.

3.1.3 React

React è la libreria su cui si basa Next.js per la costruzione dell'interfaccia utente tramite componenti dichiarativi. Il modello a componenti consente di strutturare l'interfaccia in unità indipendenti e riutilizzabili, ciascuna responsabile del proprio stato visivo e del proprio ciclo di vita.

3.1.4 Zustand

Zustand è una libreria leggera per la gestione dello stato globale dell'applicazione. A differenza di soluzioni più elaborate come Redux, non richiede boilerplate e si integra in modo naturale con TypeScript. Il middleware `persist` consente di sincronizzare automaticamente gli store con il localStorage del browser, rendendo lo stato della partita sopravvivente al ricaricamento della pagina senza codice aggiuntivo.

3.1.5 Tailwind CSS

Tailwind CSS è un framework CSS *utility-first*: invece di definire classi semantiche personalizzate, si compone l'aspetto visivo applicando direttamente nel markup classi predefinite di basso livello. Questo approccio ha velocizzato lo sviluppo dell'interfaccia, garantendo coerenza visiva e semplificando la gestione dei layout responsivi.

3.2 Strumenti di qualità del codice

3.2.1 Biome

Biome è uno strumento che unifica in un unico pacchetto le funzionalità di linting, formattazione e organizzazione degli import, sostituendo la tradizionale combinazione ESLint + Prettier con una configurazione singola e tempi di esecuzione notevolmente inferiori. È configurato per essere eseguito automaticamente a ogni commit tramite Husky.

3.2.2 Husky e Commitlint

Husky gestisce i Git hooks, permettendo di eseguire automaticamente controlli prima che un commit venga registrato. Nel progetto è configurato per eseguire Biome (linting e formattazione) e Commitlint (validazione del messaggio di commit) a ogni `git commit`. Commitlint verifica che i messaggi rispettino la convenzione *Conventional Commits*, migliorando la leggibilità della cronologia del progetto.

3.3 Strumenti di testing e documentazione

3.3.1 Vitest

Vitest è un framework di test unitari progettato per integrarsi con l'ecosistema Vite/-Next.js. Offre prestazioni superiori rispetto a Jest grazie all'esecuzione nativa dei moduli ES e supporta la misurazione della copertura del codice tramite Istanbul. Viene utilizzato per testare la logica del domain model in isolamento dall'interfaccia.

3.3.2 Playwright

Playwright è un framework per test end-to-end che automatizza browser reali (Chromium, Firefox, WebKit). Permette di verificare il comportamento completo dell'applicazione simulando le interazioni di un utente reale, coprendo scenari che i test unitari non possono rilevare.

3.3.3 Storybook

Storybook è uno strumento per lo sviluppo e la documentazione dei componenti UI in isolamento. Ogni componente è associato a una o più *stories* che ne mostrano i diversi stati e configurazioni, facilitando lo sviluppo visivo indipendente e costituendo una documentazione interattiva dell'interfaccia.

Capitolo 4

Implementazione

4.1 Specifiche funzionali

L'applicazione implementa una versione digitale del gioco da tavolo *La Città degli Imprevisti*, pensata per essere utilizzata in sessioni di gruppo guidate da un educatore o terapista del CFPIL di Varese.

4.1.1 Configurazione della partita

Prima di avviare una partita, il sistema consente di:

- impostare il numero di giocatori (da 2 a 10), ciascuno identificato da un nome e un colore assegnato automaticamente;
- definire il numero di caselle del tabellone (da 10 a 100);
- scegliere quali tipologie di caselle speciali includere tra: Mimo, Quiz, Parole alle spalle, Disegno dettato, Emozioni in musica, Indovina l'emozione, Cosa faresti se…, Test fisico, Battaglia, Caselle movimento;
- regolare la percentuale di caselle speciali rispetto al totale delle caselle normali.

La configurazione viene persistita in locale tramite il middleware `persist` di Zustand, così da essere disponibile anche in caso di ricaricamento della pagina.

4.1.2 Svolgimento del turno

Il turno di ogni giocatore segue la sequenza:

1. Il giocatore attivo lancia il dado (valore casuale 1–6).

2. La pedina si sposta lungo il tabellone del numero di caselle corrispondente al risultato del lancio.
3. Se la casella di arrivo è normale, il turno termina e passa al giocatore successivo.
4. Se la casella è speciale, viene mostrata una finestra modale con la sfida corrispondente.
5. L'educatore valuta l'esito e registra il risultato; il sistema applica le conseguenze (avanzamento, retrocessione o salto del turno).
6. In caso di collisione tra due pedine sulla stessa casella, si attiva automaticamente la meccanica di Battaglia.

4.1.3 Salvataggio e ripristino della partita

Lo stato completo della partita (posizioni delle pedine, turno corrente, stato dei mazzi) è serializzabile in formato JSON ed esportabile come file. Una partita interrotta può essere ripristinata caricando il file salvato dalla schermata dedicata.

4.2 Descrizione funzionale

L'applicazione si articola in sei schermate principali. La navigazione avviene tramite un header persistente che espone due elementi: un link diretto alla schermata delle istruzioni e un pulsante Menu che apre un overlay modale con i collegamenti a Nuova partita, Continua partita, Carica partita e Feedback.

4.2.1 Schermata Home — Configurazione della partita

La schermata iniziale permette di configurare una nuova partita: numero e nomi dei giocatori, lunghezza del percorso, tipologie di caselle speciali da includere e relativa percentuale di distribuzione.

È disponibile anche una **modalità avanzata** che consente di definire manualmente la disposizione delle caselle (descritta nella sezione successiva).

Una volta completata la configurazione, il sistema genera il tabellone e avvia la partita.

4.2.2 Schermata di gioco

È la schermata principale, composta da tre aree:

- **Tabellone**: griglia di caselle numerate, identificate visivamente per tipologia; le pedine si animano al momento dello spostamento.

- **Barra laterale**: mostra il giocatore attivo, il numero del round e le posizioni di tutti i giocatori.
- **Dado e modali**: un pulsante permette al giocatore attivo di lanciare il dado; se la casella di arrivo è speciale, si apre la finestra modale corrispondente alla sfida.

Le sfide attivabili sono:

- **Mimo** — Il giocatore mima una parola o frase senza parlare; allena la comunicazione non verbale e la capacità di decodificare i gesti altrui.
- **Quiz** — Viene posta una domanda a risposta aperta o a scelta multipla; stimola l'attenzione e il ragionamento.
- **Parole alle spalle** — Il giocatore traccia con il dito una parola sulla schiena di un compagno, che deve riconoscerla; promuove la comunicazione tattile.
- **Disegno dettato** — Un giocatore descrive a parole un'immagine che un compagno deve riprodurre graficamente; esercita la comunicazione verbale precisa e la collaborazione.
- **Emozioni in musica** — Il giocatore associa un brano musicale a un'emozione; sviluppa il riconoscimento e l'espressione emotiva.
- **Indovina l'emozione** — Viene mostrata l'immagine di un volto; il giocatore deve identificare l'emozione espressa, abilità spesso difficoltosa nei soggetti con autismo.
- **Cosa faresti se…** — Il giocatore risponde a uno scenario ipotetico di vita quotidiana; stimola il ragionamento sociale e la prospettiva altrui.
- **Test fisico** — Viene proposta una prova motoria; favorisce la coordinazione e la gestione del corpo nello spazio condiviso.
- **Battaglia** — Si attiva quando due pedine si trovano sulla stessa casella; i due giocatori si sfidano e il perdente retrocede, introducendo la gestione regolamentata della competizione.
- **Caselle movimento** — Spostamento diretto senza sfida; replica la meccanica classica del Gioco dell'Oca, introducendo l'elemento della fortuna e dell'attesa.

4.2.3 Schermata Istruzioni

Presenta le regole del gioco strutturate in sezioni (introduzione, configurazione, svolgimento del turno, caselle speciali, fine della partita), navigabili tramite un indice interno con link di ancoraggio. Ogni sezione è corredata da video dimostrativi che mostrano l'applicazione in funzione nei principali scenari di gioco. La schermata è raggiungibile direttamente dall'header in qualsiasi momento, senza perdere lo stato della partita in corso.

4.2.4 Schermata Modalità avanzata

Consente di assegnare manualmente il tipo a ogni singola casella del tabellone, sostituendo la distribuzione casuale.

4.2.5 Schermata Ripristina partita

Consente di caricare una partita precedentemente salvata tramite file JSON, ripristinando lo stato completo dal punto in cui era stata interrotta.

4.2.6 Schermata Feedback

Contiene un modulo per inviare segnalazioni o suggerimenti sull'applicazione.

4.3 Architettura dei moduli

L'applicazione segue un'architettura a tre strati con separazione netta tra logica di dominio, gestione dello stato e presentazione:

Strato	**Modulo**
Presentation Layer	Componenti React (`src/app/`)
State Management Layer	Store Zustand (`src/store/`)
Domain Model Layer	Logica di gioco (`src/model/`)

Tabella 1: Architettura a strati dell'applicazione.

Questa separazione consente di testare la logica di gioco indipendentemente dall'interfaccia e di aggiornare i componenti visivi senza modificare le regole di gioco.

4.3.1 Domain Model

Il domain model contiene tutta la logica di gioco, completamente slegata da React e dal browser. La classe `Game` è l'orchestratore principale: mantiene i riferimenti a tutti i sottocomponenti (tabellone, giocatori, dado, mazzi, manager) ed espone i metodi pubblici che il layer di stato chiama per far avanzare la partita. Il metodo `playTurn()` esegue un turno completo — lancio del dado, spostamento della pedina, elaborazione della casella — e restituisce il tipo di azione da mostrare all'utente. I metodi `toJSON()` e `fromJSON()` gestiscono la serializzazione dello stato per il salvataggio su file.

Ogni aspetto della partita è delegato a un manager specializzato:

- `TurnManager` — traccia il turno corrente, il round e il giocatore attivo;
- `MovementManager` — sposta le pedine, gestisce bonus/malus di movimento e collisioni;
- `GameStateManager` — verifica la condizione di vittoria;
- `SpecialSquareProcessor` — determina quale azione speciale si attiva su una casella;
- `BattleManager` e i manager di ogni tipologia di sfida (es. `MimeManager`, `QuizManager`) — applicano gli effetti dell'esito di ciascuna sfida.

Le caselle del tabellone sono modellate come classi distinte (es. `MimeSquare`, `QuizSquare`) che estendono la classe base `Square` e sono ricostruite tramite il pattern Builder al momento del ripristino di una partita salvata.

Ogni tipologia di casella speciale dispone di un mazzo di carte dedicato (es. `MimeDeck`, `QuizDeck`), mescolato all'inizio della partita e consumato in sequenza per garantire varietà e non ripetizione dei contenuti nella stessa sessione.

4.3.2 State Management

Lo stato globale è gestito con Zustand, suddiviso in tre store indipendenti persistiti in localStorage:

- `game-store` — stato della partita in corso: istanza serializzata di `Game`, risultato del dado, tipo di azione corrente, stato delle finestre modali;
- `config-store` — configurazione della partita: nomi e numero dei giocatori, numero di caselle, tipologie e percentuale di caselle speciali;
- `sound-store` — preferenza dell'utente riguardo agli effetti sonori.

4.3.3 Presentation Layer

Il presentation layer comprende sia le pagine che i componenti React. In Next.js le pagine (`page.tsx`) sono esse stesse componenti: la distinzione è strutturale — determinano il routing — ma non concettuale.

Le pagine dell'applicazione sono:

- `app/page.tsx` — schermata di configurazione della partita;
- `app/game/page.tsx` — schermata principale di gioco;
- `app/instructions/page.tsx` — istruzioni di gioco con video dimostrativi;
- `app/advanced-mode/page.tsx` — configurazione avanzata del tabellone;
- `app/restore-game/page.tsx` — ripristino partita da file;
- `app/feedback/page.tsx` — modulo di feedback.

I componenti principali dell'interfaccia sono:

- `board.tsx` — tabellone come griglia CSS con numero di colonne variabile in base alla dimensione;
- `square.tsx` — singola casella con numero, tipo visivo e pedine presenti;
- `pawn.tsx` — pedina con colore assegnato e animazione al momento dello spostamento;
- `dice.tsx` — dado con animazione al momento del lancio;
- `left-bar.tsx` — barra laterale con turno attivo, round corrente e classifica;
- `turn-result-modal.tsx` — finestra modale delegata a un sottocomponente specifico per ogni tipo di sfida.

I componenti primitivi riutilizzabili (`Button`, `Input`, `Label`, `Select`) sono sviluppati e documentati in isolamento tramite Storybook.

4.4 Scelte implementative rilevanti

4.4.1 Il tipo `GameActionResult` e il Command Pattern

La comunicazione tra il domain model e il layer di stato avviene tramite il tipo `GameActionResult`, un oggetto TypeScript che `playTurn()` restituisce al termine di ogni turno. Questa scelta riflette il *Command Pattern*: invece di eseguire direttamente un'azione sull'interfaccia, il domain model produce un oggetto che *descrive* l'azione da compiere, lasciando al chiamante la responsabilità di decidere come gestirla.

Il campo `type` è un'unione discriminata di stringhe letterali — una per ogni tipologia di sfida più il valore `"none"` per i turni senza azione speciale. Il campo `data` trasporta i dati specifici dell'azione (ad esempio la carta estratta o i giocatori coinvolti in una battaglia). Lo store legge `type` per determinare quale finestra modale aprire, e `data` per popolarla con il contenuto corretto, senza che il domain model abbia alcuna dipendenza dall'interfaccia. Il ciclo si chiude quando l'educatore registra l'esito della sfida: lo store chiama il corrispondente metodo `resolve*()` su `Game`, che applica le conseguenze sul tabellone e aggiorna lo stato della partita.

Listing 4.1: Tipo di ritorno di `playTurn()` (src/model/managers/types.ts)

```
export type GameActionResult = {
  type:
    | "battle" | "mime"    | "quiz"
    | "backwrite"          | "face-emotion"
    | "music-emotion"      | "physical-test"
    | "what-would-you-do"  | "dictation-draw"
    | "none";
  data?: Battle | Mime | Quiz | BackWrite | FaceEmotion
       | MusicEmotion | PhysicalTest | WhatWouldYouDo
       | DictationDraw;
  diceResult: number;
  actionType: string | null;
};
```

Il metodo `playTurn()` in `Game` sfrutta questo tipo per incapsulare l'intera logica del turno — gestione del turno saltato, lancio del dado, movimento, rilevamento collisioni ed elaborazione della casella speciale — restituendo un valore strutturato che il layer di stato consuma senza dover conoscere i dettagli interni.

Listing 4.2: Metodo `playTurn()` (src/model/game.ts)

```
playTurn(): GameActionResult {
  const currentPlayer = this.turnManager.getCurrentPlayer();

```

```
  if (currentPlayer.mustSkipTurn()) {
    this.turnManager.advanceTurn();
    return { type: "none", diceResult: 0, actionType: null };
  }

  const diceValue = this.dice.roll();
  const result = this.executePlayerTurn(currentPlayer, diceValue);
  this.turnManager.advanceTurn();

  return { ...result, diceResult: diceValue };
}
```

4.4.2 Gestione dello stato con Zustand e persistenza

Lo store `useGameStore` è il punto di raccordo tra il domain model e l'interfaccia utente. È definito tramite `create()` di Zustand, avvolto nel middleware `persist`, e si articola in due sezioni distinte: lo *stato* e le *azioni*.

La sezione di stato contiene due categorie di campi: quelli che riflettono la partita in corso (`game`, istanza della classe `Game`, e `gameData`, la sua rappresentazione JSON) e quelli che descrivono la situazione dell'interfaccia (`isModalOpen`, `isDiceModalOpen`, `diceResult`, `actionType`, `actionData` e altri).

Le azioni sono funzioni che i componenti React chiamano in risposta agli eventi dell'utente. La più importante è `playTurn()`: recupera l'istanza di `Game` dallo store, invoca il metodo omonimo del domain model, riceve il `GameActionResult` e aggiorna lo stato dell'interfaccia di conseguenza — impostando il risultato del dado, il tipo di azione attiva e aprendo la finestra modale appropriata. Analogamente, ciascuna azione `resolve*()` delega la risoluzione al metodo corrispondente di `Game` e, al termine, aggiorna la serializzazione JSON della partita tramite `toJSON()`.

Il middleware `persist` sincronizza automaticamente lo store con il localStorage del browser. L'opzione `partialize` limita la persistenza al solo campo `gameData`: in questo modo lo stato della partita sopravvive al ricaricamento della pagina, mentre i campi transitori dell'interfaccia vengono reinizializzati ai valori di default.

Listing 4.3: Store di gioco con persistenza selettiva (src/store/game-store.ts)

```
export const useGameStore = create<GameStore>()(
  persist(
    (set, get) => ({
      game: null,
      gameData: null,
      isModalOpen: false,
      // ...altri campi UI...
```

```

      actions: {
        playTurn: () => {
          const { game } = get();
          if (!game) return;

          const { diceResult, actionType, data } = game.playTurn();
          // aggiorna lo stato UI in base al risultato...
        },
        // ...altri metodi...
      },
    }),
    {
      name: "game-store",
      partialize: (state) => ({
        gameData: state.gameData,
        hasSavedGame: !!state.gameData,
      }),
    },
  ),
);
```

4.5 Design pattern utilizzati

La progettazione dell'applicazione ha fatto uso di diversi design pattern consolidati [4], applicati in modo coerente con le responsabilità dei moduli descritti nelle sezioni precedenti.

4.5.1 Command Pattern

Il pattern Command è applicato nella comunicazione tra il domain model e il layer di stato tramite il tipo `GameActionResult` (si veda la sezione 4.4.1): ogni turno produce un oggetto che *descrive* l'azione da eseguire, disaccoppiando il mittente (il domain model) dal ricevente (lo store e l'interfaccia).

4.5.2 Facade Pattern

La classe `Game` agisce da *Facade*: espone un'interfaccia semplice e coerente (`playTurn()`, `resolve*()`, `toJSON()`) nascondendo la complessità interna dei manager, dei mazzi e del tabellone. I componenti React e lo store non interagiscono mai direttamente con i sottosistemi interni.

4.5.3 Builder Pattern

Il modulo `square-builder.ts` implementa il pattern Builder per ricostruire le istanze delle caselle a partire dalla loro rappresentazione JSON durante il ripristino di una partita salvata. Poiché ogni tipologia di casella è una classe distinta, il Builder centralizza la logica di selezione del costruttore corretto in base al campo `type`.

4.5.4 Observer Pattern

Zustand adotta implicitamente il pattern Observer: i componenti React si *sottoscrivono* a porzioni specifiche dello store tramite selettori, e vengono ri-renderizzati automaticamente solo quando i campi osservati cambiano. Questo evita aggiornamenti superflui dell'interfaccia e mantiene i componenti disaccoppiati dalla logica di aggiornamento dello stato.

4.5.5 Manager Pattern

La classe `Game` non implementa da sola tutta la logica di gioco, ma delega ciascuna responsabilità a un manager specializzato: `TurnManager` per la gestione dei turni, `MovementManager` per gli spostamenti e le collisioni, `GameStateManager` per la verifica della vittoria, `SpecialSquarePro` per l'attivazione delle sfide, e un manager dedicato per ogni tipologia di sfida (es. `MimeManager`, `QuizManager`). Questo approccio — talvolta chiamato *Manager Pattern* o *Specialist Pattern* — consente di mantenere la classe `Game` snella e leggibile, concentrando ciascuna regola di gioco nel modulo che ne è responsabile, con benefici diretti sulla testabilità e sulla manutenibilità del codice.

4.5.6 Model-View-Controller

L'architettura a tre strati dell'applicazione rispecchia il pattern architetturale *Model-View-Controller* (MVC): il Domain Model (`src/model/`) corrisponde al *Model*, i componenti React (`src/app/`) alla *View* e gli store Zustand (`src/store/`) al *Controller*, che media le interazioni dell'utente verso il modello e propaga gli aggiornamenti di stato verso l'interfaccia.

Capitolo 5

Test

Capitolo 6

Conclusioni

Bibliografia

[1] Luca Surian. *L'autismo - Conoscerlo e affrontarlo.* Il Mulino, 2021.

[2] CFPIL sito ufficiale. `https://agenziaformativa.va.it/sedi/varese-cfpil/`, 2026.

[3] Gabriele Mari. Gioco strutturato, gioco modificato: un'esperienza di giochi da tavolo con bambini e ragazzi con autismo. *L'integrazione scolastica e sociale*, 17(4):315–321, 2018.

[4] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley, 1994.

?

Progetto sviluppato presso il Laboratorio di Ricerca
in collaborazione con l'Azienda Specifica
https://di.unimi.it/it/ricerca/risorse-e-luoghi-della-ricerca/laboratori-di-ricerca